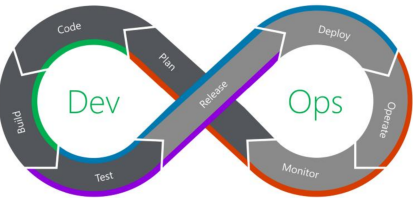


1 General Definitions

NIST Cloud: On-demand self-service, Broad network access, Resource pooling (multi-tenant), Rapid elasticity, Measured service (pay-as-you-go).

Cloud Operations: manage/optimize cloud services & infra. **GitOps:** Git = single source of truth; declarative, versioned CI/CD. **DevOps:** dev+ops; automation, CI/CD, monitoring, agile. **CI:** every push auto-build & tested. **Platform Engineering:** internal self-service dev platforms (IDP); cultural mindset. **DevOps 6 pillars:** Cultural mindset, Automation-first, Continuous Processes, Monitoring/Feedback, Rapid+Reliable Releases, Security.

1.1 DevOps Cycle



Aracs → **stages:**
CI=Code+Build+Test, **CD**=Deploy+Test+Release (manual prod), **CF**=Feed-back=Operate+Monitor+Plan.
Stages (activities | tools): **Plan** (requirements, roadmap, arch sketch, risk/rollback | Jira, Asana, GitHub Issues, Trello), **Code** (write/refactor, local build/test, peer review | GitLab, GitHub, Bitbucket), **Build** (compile, container images, publish artifacts, versioned packages | GitLab CI, GH Actions, Jenkins, Maven/Gradle, Docker/Podman, Artifactory, Quay), **Test** (unit/integration/E2E, perf/load, sec scans, coverage, quality gates | JUnit/py-test, Selenium/Cypress/Playwright, JMeter/k6, Snky/Trivy, SonarQube), **Release** (promote artifact, immutable version, release notes, signing | Cosign, GnuPG, GitLab/GH Packages), **Deploy** (pull artifact, apply manifest, strategy, post-deploy validation, rollback | ArgoCD, Flux, Helm, Kustomize, Terraform/OpenTofu, Ansible, Flagger), **Operate** (troubleshoot, scale, patch, secret rotation, drift detect, backups, com-

pliance | PagerDuty, Vault, Velero, HPA/VPA, Falco), **Monitor** (SLI/SLO, dashboards, alerts, synthetic checks, correlate logs/metrics/traces | Prometheus, Grafana, Loki, Jaeger, ELK, Alertmanager).

Delivery vs Deployment: Deployment auto-pushes staging→prod; Delivery manual gate. **CD Deployment Strategies:**

- **Rolling:** gradual update, minimal downtime
- **Blue-Green:** two envs, fast switch
- **Canary:** small user group first
- **Feature Flag:** deploy, toggle later
- **Dark Launching:** real traffic, hidden from users

1.2 Platform Engineering

Core Responsibilities: enable self-service, standardize runtimes/tooling, abstract infra, enforce compliance & security, provide observability.

8 Building Blocks (top→bottom): Self-Service Portal (IDP), Cost & Capacity, Observability, Policy & Security, Package & Artifact Mgmt, CI/CD Pipelines & Deployment, Infrastructure as Code, Compute & Orchestration.

Top Container Challenges (CNCF 2025): Cultural changes (47%), Lack of training (36%), Security (36%), CI/CD (35%), Monitoring (35%), Complexity (34%), Scaling, Networking, Logging, Service Mesh.

2 GitLab Pipelines

2.1 Core Concepts & DSL Mapping

Pipeline = automated steps → faster, safer, reliable. Traceable logs, parallel jobs, consistent envs kill "works on my machine".

Concept	Keyword	Example
Stages	stages:	[build, test, deploy]
Jobs	top-level key	<name>: {stage: script}
Steps	script:	[cmd1, cmd2]
Rules	rules/only/except	[{if:..., when:...}]
Services	services:	[{name: postgres, tags: (UI config)}]
Runners	tags:	[mmy-runner]
Caches	cache:	{key, paths}
Artifacts	artifacts:	{paths, expire_in}
Registries	script: push	docker login
Variables	variables:	{FOO: bar}

5-stage flow: commit/MR/tag → build (compile, image, package) → test (unit/integration/security/perf) → release (publish to registry) → deploy (target env) → post-deploy (smoke, health, rollback).

2.2 Jobs, Stages & Rules

.gitlab-ci.yml YAML DSL. **Job** = smallest unit, belongs to one **stage**,

runs script on a runner. Same stage → parallel; stages → sequential.

rules: when a job runs (modern only/except). **needs:** DAG, start before stage finishes. **dependencies:** which jobs' artifacts to pull (no ordering). **workflow:rules:** gate the whole pipeline.

```
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH
      when: never
  deploy:
    stage: deploy
    needs: [build] # DAG, skip stage wait
    dependencies: [build] # pull build's artifacts
    rules:
      - if: '$CI_COMMIT_BRANCH == "main"'
      - when: never
```

2.3 Cache, Artifacts & Services

Cache: shared across jobs/pipelines, speed up repeat work (deps, compiled objects). **Artifacts:** files persisted to GitLab for downstream stages or download; declare via dependencies/needs. **Services:** sidecar containers (DB, dind, API) reachable via alias.

```
integration-test:
  services: [{name: postgres:17, alias: db}]
  variables: {POSTGRES_DB: app, POSTGRES_USER: u, POSTGRES_PASSWORD: p}
  script: [psql -h db -U $POSTGRES_USER -d $POSTGRES_DB -c "SELECT 1"]
  cache: {key: deps, paths: [cache/]}
  artifacts: {paths: [bin/], expire_in: 1h}
```

2.4 Environments

Where code is deployed. **Tiers:** Local (workstation) → Dev (sandbox) → Integration (CI target) → Test/QC (interface tests) → Staging (prod mirror) → Production (live). Linkable to K8s cluster (set up via UI). environment.on_stop unstages; action: stop stops it.

2.5 Push vs. Pull Deployment

Push: (Jenkins/GitLab calls cluster): + easy, flexible targets (VMs/containers, not just K8s); - firewall open, admin access out, pipeline per env. **Pull:** (agent in cluster, e.g. ArgoCD) secure (no inbound), auto-reconcile, scales identically; - agent per cluster, polling latency.

2.6 GitLab Runner

Worker executing jobs. **Scope:** Instance (shared, GitLab-managed) / Group / Project (dedicated). **Executors:** shell, ssh, docker, docker+machine, kubernetes, virtualbox, parallels. Register: gitlab-runner register with URL + token + tags + executor. Jobs pick runner by tags:.

2.7 More Keywords & Docker Build

default: fallback
image/before_script.
before_script/after_script wrap every script. rules:changes: run only if files changed.

Predefined vars:

`$CI_REGISTRY[ IMAGE], $CI_REGISTRY_USER/ PASSWORD, $CI_COMMIT_BRANCH, $CI_COMMIT_SHORT_SHA, $CI_PROJECT_DIR, $CI_JOB_TOKEN (auth API/registry), $CI_MERGE_REQUEST_IID. Pin images by digest (img@sha256:..), not :latest → flaky.`

```
build-image:
  stage: release
  image: docker:28
  services:
    - {name: docker:28-dind, alias: docker} # Docker-in-Docker
  variables: {IMG: $CI_REGISTRY_IMAGE: $CI_COMMIT_SHORT_SHA}
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $IMG -f Dockerfile.prod .
    - docker push $IMG
  rules:
    - changes: [Dockerfile.prod]
```

3 Terraform

IaC: managing/provisioning infra through code instead of manual processes. **Declarative** (TF, what/desired state, implicit flow, stateful, idempotent) vs **Imperative** (shell/CLI, how/steps, explicit, stateless, not idempotent). TF talks to Provider plugins (RPC) -> Client SDK (HTTP) -> cloud API. **Resources** (resource) = managed objects. **Data sources** (data) = read-only lookups. HCL sample:

```
terraform {
  required_version = ">= 1.0.0"
  required_providers { aws = { source = "hashicorp/aws", version = ">= 5.0" } }
}
provider "aws" { region = "us-east-1" }
variable "instance_type" { default = "t2.micro" }
resource "aws_instance" "web" {
  ami = data.aws_ami.ubuntu.id
  instance_type = var.instance_type
}
output "public_ip" { value = aws_instance.web.public_ip }
```

Workflow: write .tf -> init (backend + download providers/modules) -> plan -> apply -> destroy. Other: fmt, validate, console (REPL), state list/show <addr>. Reference attrs as <type>.<name>.<attr>.

3.1 OpenTofu

Fork of Terraform v1.5.7 under **Linux Foundation**, MPL-2.0 (TF switched to BUSL). Drop-in replace-

ment, adds **state encryption**. Avoids single-vendor lock-in.

3.2 HCL Features

count (numeric loop, count.index) and **for_each** (map/set loop, each.key/value) for multiple instances. **Data source** example:

```
data "aws_ami" "ubuntu" {
  most_recent = true
  filter { name = "name" values = ["ubuntu/images/hvm-ssd/ubuntu-*"] }
  filter { name = "virtualization-type" values = ["hvm"] }
  owners = ["099720109477"] # Canonical
}
```

3.3 Variables

Precedence: CLI -var/-var-file > env TF_VAR.<name> > *.tfvars > default. No default & not supplied -> CLI prompts operator. TF_VAR_ prefix ideal for secrets (TF_VAR_access_key=...). depends_on forces explicit ordering; implicit deps come from attribute references.

3.4 State

terraform.tfstate (JSON) maps HCL to real resource IDs (e.g. ARN). **3-way diff** on apply: **NEW** (HCL) vs **PREVIOUS** (.tfstate) vs **EXISTING** (cloud refresh) -> decides actions, detects drift. **Remote backend** (S3) shares state in teams; **state locking** via use_lockfile (S3-native, replaces DynamoDB):

```
terraform {
  backend "s3" {
    bucket = "my-tfstate"
    key = "prod/terraform.tfstate"
    region = "us-east-1"
    use_lockfile = true
  }
}
```

3.5 File & Folder Layout

Files: terraform.tf (required_version/providers), backend.tf (remote state, kept out of VCS), providers.tf (auth/config), main.tf (resources/data), variables.tf (inputs, alphabetical), outputs.tf (returns, alphabetical), locals.tf (derived values), override.tf (loaded last, sparingly).

One folder per env (dev/, prod/) -> separate state files so a corruption hits only one env; shared modules/ alongside.

3.6 Modules

Reusable folder of .tf files (root config is itself a module):

```
module "vpc" { source = "../modules/vpc" cidr = "10.0.0.0/16" }
# reference output: module.vpc.vpc_id
```

3.7 Patterns & Recovery

import adopts existing infra; terraform apply -replace=<addr> (or legacy taint) forces a destroy+recreate of a single resource. Region-aware AMI via map:

```
import { to = aws_instance.web id = "i-0abc123" }
# then: apply

variable "AMIS" { type = map(string) default = {
  us-east-1 = "ami-05b1..." } }
ami = var.AMIS[var.region]
```

4 Ansible

Configures provisioned machines (install/configure software, deploy code); *after* provisioning. Idempotent (no-op if already in desired state), agentless, push-based, no statefile/locking. Needs Python on host & targets. Other CM tools: **Puppet**, **Chef**, **CFEngine**, **Juju**. **TF vs Ansible**: TF = provisioning, declarative, state, idempotent. Ansible = CM, *hybrid* (imperative tasks, declarative modules), no lifecycle, idempotency *optional*. Both: templating, agentless, OSS.

Execution: over SSH, copies generated Python module to ~/.ansible/tmp on target, runs with remote Python, returns JSON on stdout, deletes. Nothing stays installed. **Exceptions**: raw module runs cmd via SSH (no Python needed, e.g. bootstrap); connection: local runs on controller (network devices); Windows via **WinRM**.

4.1 Initial Setup & Inventory

ssh-keygen -t ed25519 -C *@y" on controller, ssh-copy-id user@server, test with ansible all -m ping. **Inventory** (INI or YAML) lists managed hosts grouped, referenced by play's hosts:. ansible-inventory -i inv --list verifies it.

```
[web]
web1.example.com
web2.example.com  ansible_host=10.0.0.5
[web:vars]
ansible_user=deploy
```

4.2 Playbooks, Plays & Tasks

YAML file = playbook (1..n plays); play = hosts + tasks; task = one module call. Run: ansible-playbook -i inv playbook.yml. Flags: --check (dry run), --list-tasks (preview), --tags/--skip-tags, --ask-vault-pass.

```
- name: Example
hosts: web
become: true # sudo
gather_facts: false # skip setup (faster)
force_handlers: true # run handlers even if a
task fails

vars:
  packages: [nginx, curl]
```

```
secret: "{{ vault_password }}"
roles:
  - myrole
tasks:
  - name: Install packages # list-as-option >
    loop (faster, deps OK)
    ansible.builtin.apt: # FQCN: namespace.
      collection: module
      name: "{{ packages }}"
      state: present
  notify: restart nginx
  - name: Render config
    ansible.builtin.template: { src: app.conf.
      j2, dest: /etc/app.conf }
  when: enable_service
  tags: config
handlers:
  - name: restart nginx
    ansible.builtin.service: { name: nginx,
      state: restarted }
```

4.3 Ad-hoc, Modules & Errors

ansible -i inv -m <mod> -a '<args>' <target>. setup = gather facts (ansible_facts). command/shell/raw *not* idempotent; shell allows pipes/vars. Playbook stops at first fail unless ignore_errors: true (use sparingly). **Handlers**: only run when notified *and* all prior tasks OK; force_handlers overrides.

```
- name: validate
  ansible.builtin.command: httpd -t
  register: out
  changed_when: out.failed
  failed_when: false
- ansible.builtin.file: { path: /data, state:
  directory } # file/link/touch/absent
- ansible.builtin.lineinfile: { path: /etc/x.
  conf, line: "Include sites/*" }
```

4.4 Vault, Collections, Roles & Tags

Vault: ansible-vault create|edit|view|encrypt foo.yml; encrypt_string 'secret' --name 'pw' inlines a single var. Pass via --vault-id <prompt>|path/file|name@script. Ciphertext safe to commit. **Collections** bundle modules/roles/plugins. FQCN ns.coll.mod (e.g. cisco.ios.ios_banner); independent versioning. Install: ansible-galaxy collection install <n> or via requirements.yml (collections: + roles:) then install -r. Sources: **Galaxy** (community), **Automation Hub** (RedHat, certified).

Roles: ansible-galaxy init <n> creates dirs tasks/ handlers/ defaults/ vars/ files/ templates/ meta/ tests/; use via roles: list. **Tags**: --tags/--skip-tags. Inherit from play/import.* (static); include_* (dynamic) needs explicit apply: tags:. Specials: *always* runs unless --skip-tags=always; *never* runs only via --tags=never.

4.5 Jinja2 Templating

Rendered on **controller**, only result sent to target. {{

inventory_hostname }} = current target. Filters: ipaddr/ipv4/ipv6 (validate/version), hash('sha1'), default(x), min/max/shuffle.

```
{% for host in groups['webservers'] %}
BalancerMember http://{{ host }}:{{ port |
  default(80) }}
{% endfor %}
```

5 Kubernetes (K8)

Object: persistent record-of-intent. **Controller**: reconciles current → desired state.

5.1 Architecture

Control plane: kube-apiserver (REST front), etcd (KV store), kube-scheduler (pod → node), kube-controller-manager (built-in controllers), cloud-controller-manager (cloud glue, optional). **Node**: kubelet (runs pods via container runtime), kube-proxy (Service networking/iptables).

5.2 Objects

Pod: smallest unit, 1+ containers share net/storage. **Sidecar**: aux container alongside main (logs, proxy). **Init container**: runs to completion before main. **ReplicaSet**: keeps N pod replicas (rarely used directly). **Deployment**: manages ReplicaSets, rolling updates, rollback. **StatefulSet**: ordered pods with stable identity + per-pod PVC (DBs, brokers). **DaemonSet**: one pod per (matching) node – logs, CNI, monitoring. **Service**: stable VIP/DNS + LB over pod set. **Ingress**: L7 HTTP(S) endpoint. **Volume**: dir mounted into pod. **ConfigMap**: non-secret KV config. **Secret**: sensitive KV, base64-encoded, *not* encrypted by default. **HPA**: autoscales replicas on metrics.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: app }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 1, maxSurge: 1 }
  template:
    metadata: { labels: { app: web } }
    spec:
      initContainers:
        - { name: init, image: busybox, command: ['sh','-c','sleep 10'] }
      containers:
        - name: web
          image: nginx
          ports: [{ containerPort: 80 }]
        - { name: sidecar, image: busybox, command:
          ['sh','-c','while true; do sleep 30; done'] }
```

5.3 Services

Stable VIP + DNS over label-selected pods, round-robin LB. **ClusterIP** (de-

fault) internal only; **NodePort** static port on every node <NodeIP>:<port>; **LoadBalancer** cloud LB w/ public IP; **ExternalName** CNAME alias to external DNS (no proxy). **Ingress** routes HTTP(S) by host/path (needs controller: Nginx, Traefik). **Gateway API** (Gateway+HTTPRoute/TLSRoute) – modern, multi-protocol, role-oriented Ingress replacement. **Port trap**: Service port vs targetPort (container) vs nodePort; Ingress backend refs Service port.

5.4 ConfigMaps & Secrets

Inject single var (configMapKeyRef/secretKeyRef), all keys (envFrom), or mount as files.

```
env:
  - name: APIKEY
    valueFrom: { configMapKeyRef: { name: cfg, key:
      api-key } }
envFrom:
  - configMapRef: { name: cfg }
volumeMounts:
  - { name: v, mountPath: /etc/cfg }
volumes:
  - { name: v, configMap: { name: cfg } } # or:
    secret: { secretName: s }
```

5.5 Storage: PV, PVC, StorageClass

Container FS is ephemeral. **Volume**: dir in pod, lives w/ pod (emptyDir, hostPath, NFS, ...). **PersistentVolume (PV)**: cluster-scoped piece of external storage. **PersistentVolumeClaim (PVC)**: namespaced request for storage (size, accessMode); binds to a matching PV. **StorageClass**: dynamic provisioner – creates a PV on-demand if no static PV matches (e.g. gp3, standard-rwo).

5.6 LimitRange & ResourceQuota

LimitRange: per-container min/max + default req/lim in NS; out-of-range Pod rejected, missing req/lim auto-injected. **ResourceQuota**: caps summed NS usage; over-quota Pod rejected.

5.7 Namespaces

Logical isolation in one cluster (RBAC, quotas). Namespaced (Pods, Services) isolated; cluster-wide (Nodes, PV) shared. Defaults: default, kube-system, kube-public, kube-node-lease.

5.8 Rolling Updates

Deployment ensures availability during change. **Recreate**: kill all → recreate (downtime). **RollingUpdate** (default): tuned by maxUnavailable (max pods down) + maxSurge (max extra pods). kubectl rollout history/undo for revisions.

5.9 Scheduling

kube-scheduler picks node via **Filtering** then **Scoring**. *Filtering* (feasibility): PodFitsHostPorts, PodFitsResources, PodMatchNodeSelector, CheckNodeDiskPressure, CheckVolumeBinding. *Scoring* (rank): SelectorSpreadPriority (spread same Service/STS/RS), LeastRequestedPriority (free resources), NodeAffinityPriority.

```
spec:
  nodeSelector: { disktype: ssd }
```

Taints on nodes repel pods; **Tolerations** on pods allow (don't force) them onto tainted nodes – pair w/ nodeSelector/affinity to pin. Effects: *NoSchedule*, *PreferNoSchedule*, *NoExecute* (evict). No effect on DaemonSets.

CLI: kubect1 taint nodes <n> gpu=true:NoSchedule, kubect1 label nodes <n> disktype=ssd (trailing – removes).

```
tolerations:
- { key: "gpu", operator: "Equal", value: "true",
  effect: "NoSchedule" }
```

cordon/drain: kubect1 cordon <n> marks unschedulable (existing stay); drain <n> also evicts pods (maintenance); uncordon <n> reverts.

5.10 Probes & Resources

kubelet probes via httpGet/tcpSocket/exec. **Liveness**: fail → restart. **Readiness**: fail → remove from Service endpoints. **Startup**: guards slow boots, disables others until OK. Repeated liveness fail → exp backoff → CrashLoopBackOff. **requests**: reserved min (scheduling). **limits**: ceiling – CPU throttled, memory over-limit → OOMKilled.

```
resources:
  requests: { cpu: "250m", memory: "64Mi" }
  limits: { cpu: "500m", memory: "128Mi" }
livenessProbe:
  httpGet: { path: /healthz, port: 80 }
  initialDelaySeconds: 5
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /ready, port: 80 }
```

5.11 Commands

Apply: kubect1 {create|apply|replace} -f manifest.yaml
Gen YAML: kubect1 create deploy web --image=nginx --dry-run=client -o yaml > d.yaml
Imperative: kubect1 run, scale deploy x --replicas=4, expose Update image: kubect1 set image deploy/x web=nginx:1.27
Autoscale: kubect1 autoscale